

Hate Speech Detection System

A Bidirectional LSTM Deep-Learning Classifier for Text & Memes
(English and Roman Urdu) with a PaddleOCR v4 Image Pipeline

Project Report & Technical Explanation

Prepared for academic review

Technology stack: Python · TensorFlow / Keras · Streamlit · PaddleOCR · scikit-learn

1. Project Overview

This project is an interactive **Hate Speech Detection System** that reads a piece of text and classifies it into one of three categories: **Hate Speech**, **Offensive Language**, or **Neither**. The system is delivered as a web application built with **Streamlit**, so a user can type text or upload an image and receive an instant prediction together with the confidence for each of the three classes.

The application supports two independent input modes and two languages:

- **Text Analysis** — the user types or pastes text directly, which is fed to the classifier.
- **Image Analysis** — the user uploads a meme/image; the text is first read out of the image using Optical Character Recognition (OCR), and that extracted text is then classified.
- **English** and **Roman Urdu** are both supported, each with its own trained model.

The primary classifier is a locally-trained **Bidirectional LSTM (BiLSTM)** neural network built in Keras. Two alternative Hugging Face backends are also selectable as a comparison, so the same interface can run a zero-shot model or a pre-trained transformer classifier.

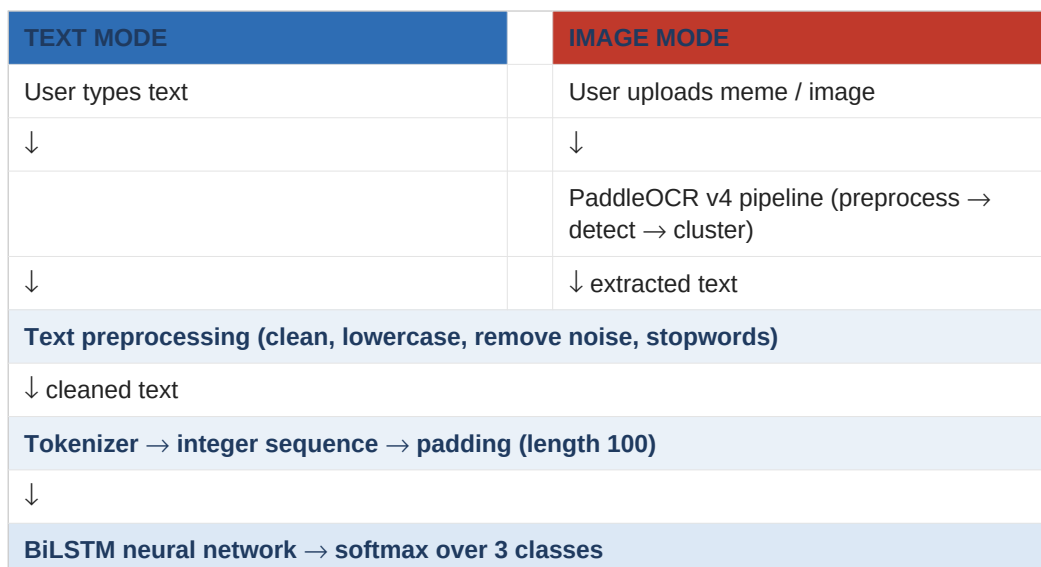
1.1 The three output classes

Class ID	Label	Meaning
0	Hate Speech	Content that promotes hatred or violence against a group of people.
1	Offensive Language	Rude, abusive or vulgar language that is not necessarily targeted hate.
2	Neither	Normal, neutral content that is neither hateful nor offensive.

This 0 / 1 / 2 mapping is fixed and used identically across training, inference and the UI.

2. System Architecture

The system is organised around one central classifier that both input modes feed into. The diagram below shows the two paths — direct text, and image-to-text via OCR — converging on the same BiLSTM prediction engine.



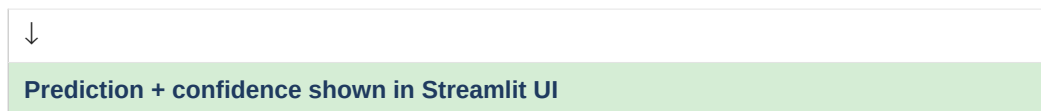


Figure 1 — Both input modes converge on the same BiLSTM classifier.

2.1 Main source files

File	Responsibility
<code>app.py</code>	Streamlit UI — sidebar controls, text/image modes, calls the models and renders results.
<code>train_lstm_model.py</code>	Trains the English BiLSTM and saves the model, tokenizer and config.
<code>train_lstm_model_roman_urdu.py</code>	Same training pipeline for the Roman Urdu dataset.
<code>lstm_inference.py</code>	LSTMPredictor class — loads the model and runs preprocessing + prediction.
<code>ocr_pipeline.py</code>	MemeOCRPipeline — PaddleOCR v4 image-to-text pipeline.
<code>huggingface_inference.py</code>	Alternative zero-shot Hugging Face backend.
<code>huggingface_text_classifier.py</code>	Alternative pre-trained transformer classifier backend.

3. Data and Text Preprocessing

The English model is trained on a labelled tweets dataset (**labeled_data.csv**) whose *tweet* column holds the text and *class* column holds the 0/1/2 label. The Roman Urdu model is trained on a multilingual dataset filtered to the Roman-Urdu rows. Because these are user-generated social-media texts, they are noisy and must be cleaned before the network can learn from them.

3.1 Preprocessing steps

Every text — during training **and** at prediction time — passes through the same cleaning routine:

- Convert the text to **lowercase**.
- Remove **URLs** (http / https / www links).
- Remove **@mentions** and the # symbol from hashtags.
- Strip **punctuation** (? : ! . , ;) and quotation marks.
- Replace tabs, newlines and repeated spaces with a single space.
- **English only**: remove the possessive 's and drop common English **stopwords** (the, is, and, ...).
- **Roman Urdu only**: remove apostrophes but **keep** stopwords — Roman Urdu has no standard stopword list and removing words could change the meaning.

Why this matters: the tokenizer assigns each word a numeric ID. If the cleaning at prediction time differed even slightly from training, the same word could map to a different ID and the model's accuracy would collapse. For this reason the preprocessing in `lstm_inference.py` is kept byte-for-byte identical to the training script.

3.2 Cleaning routine (from the code)

```
text = text.lower()
text = re.sub(r'http\S+|www\S+|https\S+', '', text)
text = re.sub(r'@\w+|#', '', text)
for p in '?!.,;': text = text.replace(p, '')
text = text.replace('\n', ' ').replace('\t', ' ').replace('\"', '')
# English: remove "'s" and NLTK stopwords
words = [w for w in text.split() if w not in stop_words]
text = ' '.join(words)
```

4. Tokenization and Sequence Encoding

A neural network cannot read words directly — it needs numbers. The Keras **Tokenizer** solves this by building a vocabulary from the training data and mapping every word to an integer:

- The tokenizer is fit on the cleaned training texts and keeps the **10,000** most frequent words.
- Any word never seen during training is mapped to a special <OOV> (out-of-vocabulary) token, so unknown words don't crash the model.
- Each sentence becomes a list of integers, e.g. *"i hate them"* → [5, 82, 34].
- Sequences are **padded / truncated to a fixed length of 100** so every input is the same shape. Padding is added at the end (*post padding*).

The fitted tokenizer is saved to `tokenizer.pkl` and reused at prediction time — this is essential, because the model only understands the exact word-to-number mapping it was trained with.

5. The BiLSTM Model — How It Works

The classifier is a **Bidirectional Long Short-Term Memory (BiLSTM)** network. An LSTM is a type of recurrent neural network that reads a sentence word-by-word while maintaining a memory of what it has seen so far, which lets it capture context and word order — something a simple bag-of-words model cannot do. A **Bidirectional** LSTM reads the sentence in **both directions** (left-to-right and right-to-left) and combines the two, so each word is understood in the context of the words both before and after it.

5.1 Layer-by-layer architecture

#	Layer	Configuration	Purpose
1	Embedding	vocab → 128-d, mask_zero=True	Turns each word ID into a dense 128-dimensional vector that captures meaning.
2	SpatialDropout1D	rate = 0.2	Randomly drops whole embedding dimensions to reduce overfitting.
3	Bidirectional LSTM	64 units each way (128 total)	Reads the sequence both ways to learn context and word order.
4	Dropout	rate = 0.3	Further regularisation before the dense layers.
5	Dense	64 units, ReLU	Learns higher-level combinations of the LSTM features.
6	Dropout	rate = 0.3	Regularisation.
7	Dense (output)	3 units, Softmax	Produces the probability of each class; the three sum to 1.

Table — The Keras Sequential model as defined in `train_lstm_model.py`.

5.2 Why each design choice was made

- **Embedding layer** — lets the network learn that words like *hate* and *despise* are related, instead of treating every word as unrelated.
- **Bidirectional** — negation and target often appear on either side of a keyword, so reading both directions improves understanding of intent.
- **Dropout & SpatialDropout** — social-media data is small and noisy, so aggressive regularisation prevents the model from memorising the training set.
- **Softmax output** — gives a clean probability for each of the three classes, which the UI displays as confidence bars.

6. Training the Model

The model is compiled with the **Adam** optimizer and **categorical cross-entropy** loss (the standard choice for multi-class classification), and trained with these settings:

Hyper-parameter	Value	Note
Vocabulary size	10,000	Most frequent words kept
Sequence length	100	Padded / truncated

Embedding dimension	128	Size of each word vector
LSTM units	64 ($\times 2 = 128$)	Per direction
Batch size	64	
Max epochs	20	Stops early if no improvement
Train / test split	70% / 30%	Stratified, random_state=42
Optimizer / Loss	Adam / Categorical cross-entropy	

6.1 Training callbacks (smart stopping)

- **EarlyStopping** — halts training when validation loss stops improving for 5 epochs and restores the best weights, preventing overfitting and wasted time.
- **ModelCheckpoint** — automatically saves the best-performing model (highest validation accuracy).
- **ReduceLROnPlateau** — lowers the learning rate when progress stalls, helping the model settle into a better solution.

After training, the script evaluates on the held-out test set, prints a full **classification report** (precision / recall / F1) and a **confusion matrix**, and saves a training-history plot. Three artefacts are produced: the model (`bilstm_model.h5`), the tokenizer (`tokenizer.pkl`) and the config (`lstm_config.pkl`).

7. Making a Prediction (Inference)

At run time the **LSTMPredictor** class (in `lstm_inference.py`) loads the three saved artefacts once and reuses them. A single prediction follows these steps:

Step	What happens
1	The input text is cleaned with the exact same preprocessing used in training.
2	If the text is empty after cleaning, the model short-circuits to a near-uniform distribution predicting 'Neither' (a safe default).
3	The tokenizer converts the cleaned text into an integer sequence.
4	The sequence is padded/truncated to length 100.
5	The BiLSTM outputs three probabilities via softmax.
6	argmax picks the winning class; its probability becomes the confidence score.
7	The label (Hate Speech / Offensive Language / Neither) and confidence are returned to the UI.

The Streamlit UI then shows a coloured verdict (red / amber / green), a confidence percentage, and a probability bar for all three classes. The same `predict()` contract — (`label`, `confidence`, [`p_hate`, `p_off`, `p_neither`]) — is implemented by all three backends, which makes them fully interchangeable behind the interface.

8. Image Analysis — the OCR Pipeline

In Image mode the app must first read the text out of a meme before it can classify it. This is handled by the **MemeOCRPipeline** (PaddleOCR v4) in four stages:

Stage	Name	What it does
0	Preprocess	Upscales small images and builds 3 views — CLAHE-enhanced colour, a colour-inverted view (so white 'Impact-font' meme text becomes dark and readable), and a bilateral-filtered grayscale view.
1	Detect	Runs PaddleOCR on all 3 views, removes duplicate detections with non-maximum suppression (NMS), and keeps only regions with confidence ≥ 0.6 . EasyOCR is an optional fallback.
2	Cluster	Uses DBSCAN on the vertical position of each text box to separate the top caption from the bottom caption, then sorts words left-to-right within each line.
3	Classify	The combined extracted text is handed to the BiLSTM, exactly like typed text.

The multi-view approach is the key idea: memes often use white text with a black outline, which ordinary OCR struggles with. By also feeding an **inverted** copy of the image, that white text becomes dark-on-light and is detected reliably.

9. Roman Urdu Support & Alternative Backends

9.1 Roman Urdu model

A second model is trained specifically for **Roman Urdu** (Urdu written in the Latin alphabet, e.g. "*islam aman ka mazhab hai*"). It uses the same BiLSTM architecture but a separate tokenizer and configuration, and importantly it **does not remove stopwords**, because Roman Urdu has no standardised stopword list and removing words risks changing the meaning. The training data is filtered from a multilingual dataset and mapped onto the same class scheme.

9.2 Hugging Face comparison backends

Two transformer-based backends can be selected instead of the local BiLSTM:

- **Zero-shot** (`valhalla/distilbart-mnli-12-1`) — classifies text against the three class names without any task-specific training.
- **Text-classifier** (`cardiffnlp/twitter-roberta-base-hate`) — a RoBERTa model pre-trained on hate-speech data; its native labels are mapped onto the project's three classes.

Because all three backends expose the identical `predict()` interface, switching between them requires no change to the rest of the app — useful for comparing the custom-trained model against strong pre-trained baselines.

10. How to Run the Project

10.1 Install dependencies

```
pip install -r requirements.txt
```

This also auto-downloads NLTK stopwords and the PaddleOCR models (~50 MB) on first run.

10.2 Train the models (produces the .h5 / .pkl artefacts)

```
python train_lstm_model.py # English BiLSTM
python train_lstm_model_roman_urdu.py # Roman Urdu BiLSTM
```

10.3 Launch the web app

```
streamlit run app.py # opens http://localhost:8501
```

On Windows the helper scripts `run_app.bat` (default browser) and `run_app_chrome.bat` (forces Chrome) do the same thing.

11. Summary — What Was Built

In short, this project delivers an end-to-end, multi-lingual hate-speech classifier:

- A cleaned, reproducible **text-preprocessing** pipeline shared exactly between training and inference.
- A **Bidirectional LSTM** classifier (Embedding → SpatialDropout → BiLSTM → Dense → Softmax) trained with early stopping, checkpointing and learning-rate scheduling.
- Full **evaluation** — accuracy, precision/recall/F1 and a confusion matrix on a held-out test set.
- A second BiLSTM for **Roman Urdu**, with language-appropriate preprocessing.
- An **image (meme) pipeline** using PaddleOCR v4 with multi-view preprocessing, NMS deduplication and DBSCAN caption separation, feeding extracted text into the same classifier.
- A polished **Streamlit** interface with text and image modes, language selection, three swappable model backends, confidence visualisation and debug panels.

Core concept in one line: clean the text the same way every time, turn words into numbers with a fixed tokenizer, let a bidirectional LSTM read the sequence both ways to understand context, and output a probability for Hate Speech / Offensive Language / Neither.